

Téma 2: Výpočetní složitost, shrnutí

(vytvořeno s podporou nástroje ChatGPT, <https://chat.openai.com/>)

Výpočetní složitost se zabývá měřením časové náročnosti algoritmu, tj. kolik času potřebuje algoritmus pro své provedení. Výpočetní složitost může být měřena v nejlepším, průměrném a nejhorším případě.

- **Nejlepší případ:** Nejlepší případ nastává, když algoritmus dosahuje nejrychlejší možné časové složitosti pro daný problém. Je to teoretický ideální případ, kdy vstupní data jsou pro algoritmus optimální. Nejlepší případ se často nepoužívá jako měřítko efektivity, ale je užitečný pro identifikaci limitních hodnot a potenciálních omezení algoritmu.
- **Průměrný případ:** Průměrný případ bere v úvahu pravděpodobnost výskytu jednotlivých vstupních dat a jejich rozdělení. Je to průměrná časová složitost algoritmu při všech možných vstupních datech. Výpočet průměrného případu vyžaduje znalost pravděpodobnostního rozdělení vstupních dat.
- **Nejhorší případ:** Nejhorší případ je situace, ve které algoritmus dosahuje nejdelší možné časové složitosti pro daný problém. Je to situace, kdy algoritmus potřebuje nejvíce času pro provedení svých operací. Nejhorší případ se obvykle používá jako nejhorší možný scénář pro plánování a odhadování výkonu algoritmu.

Složitost algoritmu se v praxi často vyjadřuje pomocí notace velkého O (Big O notation), která udává horní odhad růstu časové nebo paměťové složitosti algoritmu v závislosti na velikosti vstupních dat. Například $O(n)$ značí lineární složitost, kde růst algoritmu je přímo úměrný velikosti vstupních dat (n).

Základní třídy časové složitosti algoritmických problémů jsou:

- **P:** zahrnuje problémy, které jsou řešitelné v polynomiálním čase. To znamená, že existuje deterministický algoritmus, který vyřeší problém v časové složitosti omezené polynomem.
- **NP:** zahrnuje problémy, u kterých lze ověřit řešení v polynomiálním čase. Pokud je potenciální řešení k dispozici, může být ověřeno rychle.
- **NP-úplné problémy:** zahrnuje nejtěžší problémy ve třídě NP. Jsou to problémy, které jsou NP a mají vlastnost, že každý problém z třídy NP lze převést na něj v polynomiálním čase.

Paměťová složitost se zabývá měřením potřebného místa v paměti pro provedení algoritmu. Určuje, kolik paměti je potřeba pro uložení dat a pomocných struktur během provádění algoritmu.

Z hlediska generických programátorských řídicích struktur jsou pro časovou efektivitu algoritmů zásadní zejména následující dvě:

- Iterace, tj. opakování určitého kódu nebo akce. Používá se k procházení prvků v poli nebo seznamu. Složitost iterace závisí na počtu prvků, které je potřeba projít.
- Rekurze, tj. proces, kdy se funkce volá sama na sebe. Tím umožňuje opakující se struktury a problémy řešit elegantním způsobem. Složitost rekurze může být vyšší než u iterace kvůli nutnosti opakovaného volání funkce.

Její časovou náročnost lze minimalizovat prostřednictvím dynamického programování. To je technika, umožňující řešení složitých problémů rozdělením na menší podproblémy (rozděl a panuj) a následným kombinováním jejich řešení. Tím se snižuje celková složitost algoritmu. Složitost dynamického programování závisí na počtu podproblémů a jejich velikosti.

Pro určení asymptotické složitosti rekurzivních algoritmů se používá Master theorem, což je sada matematických formulí, poskytujících rychlou metodu jejího odhadu.